

Reviewer Pack — Minimal Frobenius Norm of Quotient Algebras and Circuit Mono...

Entry 5f38fa0ff362 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 17:41:37 UTC

Minimal Frobenius Norm of Quotient Algebras and Circuit Monotone Width

Entry ID: 5f38fa0ff362 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 17:41:37 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quotient Algebras) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every CNF formula φ with m clauses and n variables, the minimal Frobenius norm of the quotient algebra constructed from the clause indicator matrix $A(\varphi)$ modulo the ideal generated by the negation of the identity matrix is $\Theta(m^{(1/2)^n}(3/4))$.

Rationale (proposer's reasoning):

Quotient algebras can encode non-commutative aspects of Boolean functions that are not captured by traditional algebraic structures. The Frobenius norm provides a measure of the distance from the trivial representation, which may reflect the complexity of the circuit representing φ . This conjecture suggests a potential connection between non-commutative structure and circuit complexity.

Taxonomy category: c003b_cumulative_entropy/SC2#c49 (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4016cb37db9ba2e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNF formulas φ with m clauses and n variables ($n \leq 40$), the minimal Frobenius norm of the quotient algebra is within a factor of 1.5 times the square root of m multiplied by the cube root of n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Frobenius norm" AND "quotient algebras" AND "circuit monotone width" - "clause indicator matrix" AND "Frobenius norm" AND "Boolean circuit complexity" - "algebraic combinatorics" AND " $\Theta(m^{(1/2)}n^{(3/4)})$ " AND "circuit monotone width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def construct_quotient_algebra(cnf):
        n = len(set(abs(lit) for lit in sum(cnf, [])))
        A = [[0] * n for _ in range(n)]
        for clause in cnf:
            for lit1 in clause:
                for lit2 in clause:
                    i, j = abs(lit1) - 1, abs(lit2) - 1
                    if lit1 > 0 and lit2 > 0:
                        A[i][j] += 1
                    elif lit1 < 0 and lit2 < 0:
                        A[i][j] -= 1
        return A

    def frobenius_norm(A):
        n = len(A)
        norm = 0
        for i in range(n):
            for j in range(n):
                norm += A[i][j] ** 2
        return math.sqrt(norm)
```

```

def circuit_monotone_width(cnf):
    # Placeholder implementation, replace with actual calculation
    return len(cnf) / len(set(abs(lit) for lit in sum(cnf, [])))

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    m = random.randint(1, min(n * (n - 1) // 2, 100))
    cnf = generate_cnf(n, m)
    A = construct_quotient_algebra(cnf)
    norm = frobenius_norm(A)
    width = circuit_monotone_width(cnf)
    results.append((norm, width))

mean_norm = sum(norm for norm, _ in results) / len(results)
mean_width = sum(width for _, width in results) / len(results)
correlation_coefficient = 0
if len(results) > 1:
    numerator = sum((norm - mean_norm) * (width - mean_width) for norm, width in results)
    denominator = math.sqrt(sum((norm - mean_norm) ** 2 for norm, _ in results)) * math.sqrt(s)
    correlation_coefficient = numerator / denominator

conjecture_holds = abs(correlation_coefficient) > 0.5
counterexample = "" if conjecture_holds else "correlation_coefficient"

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {seed} {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc9b0a3a.py", line 90, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc9b0a3a.py", line 59, in run_trial
    A = construct_quotient_algebra(cnf)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc9b0a3a.py", line 38, in construct_quo
    A[i][j] -= 1
    ~^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that the conjecture can be properly tested.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12597
2	propose	ollama_remote	glm4:latest	0	0	14630
3	preregistration	ollama_remote	glm4:latest	0	0	9590
4	novelty	ollama_remote	glm4:latest	0	0	8775
5	novelty	ollama_remote	glm4:latest	0	0	8929
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15691
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10826
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12304
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11995
10	judge	ollama_remote	glm4:latest	0	0	21432

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126771 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5f38fa0ff362.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5f38fa0ff362.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5f38fa0ff362.tar.gz` (if generated)